JGi JAIN

FACULTY OF ENGINEERING AND TECHNOLOGY

DEEMED-TO-BE UNIVERSITY

School of Computer Science and Engineering

B. Tech CSE

Python Programming

Mini Project Report

Name	J S Rahul
USN	25BTRPW048
Class/Section	CSE-PWC-A
Subject Name	Python Programming
Faculty Name	Ms. Sujithra G
Date of Report Submission	10/09/2026
Project Topic	SmartBill – Smart Store Billing and Inventory Management System
GitHub Link	https://github.com/JSRAHUL/SmartBIll

Table of Content

S.No	Topic	Page No
1	Introduction	3
2	Problem Statement	3
3	Objectives	3
4	Project Description	4
5	Concepts Used in the Program	4–5
6	Algorithm	5
7	Flow Chart	6
8	Python Program	7–9
9	Output	10
10	Conclusion	11

1. Introduction

In retail commerce, generating accurate invoices and managing inventory are critical everyday computational tasks. Traditional billing practices that involve manual calculations and handwritten slips frequently result in computational errors, inaccurate tax applications, lost transaction history, and accidental overselling when stock levels are depleted. To address these challenges, **SmartBill** is designed and developed as a comprehensive retail billing and stock management system implemented in Python.

The application seamlessly integrates product cataloguing, shopping cart management, automatic multi-tiered discounting, itemized Goods and Services Tax (GST) computation, payment processing, real-time inventory deduction, and sales analytics. The system is implemented using clean modular functions and relational persistence (JSON and SQLite), separating business logic from user interface operations.

The program incorporates defensive input validation and robust exception handling to ensure that non-numeric entries, negative numbers, invalid mobile numbers, and insufficient inventory attempts are handled gracefully without application crashes. These features demonstrate the practical application of foundational and intermediate Python programming principles in solving a real-world commercial problem.

2. Problem Statement

Design and develop a Bill Generator application using Python that generates a customer bill for purchased products. The program should allow users to enter product details, quantity, and price, calculate the total amount, apply GST (if applicable), and display a formatted bill. The system must prevent negative inventory, support item-level and bill-value discounts, handle multiple payment methods, and maintain persistent historical records.

3. Objectives

- To develop a modular and interactive billing application using Python.
- To maintain a persistent product catalogue containing product ID, name, category, price, stock, and GST rate.
- To automatically retrieve product details and prices from the catalogue to eliminate manual cashier pricing errors.
- To implement dual-tier discounts: item-level bulk quantity discounts and overall bill-value tiered discounts.
- To calculate statutory GST accurately across varying tax slabs (5%, 12%, 18%) post-discount on the net taxable amount.
- To implement real-time inventory synchronization by automatically decrementing stock upon successful payment.
- To validate stock availability before purchase and prevent negative stock or overselling.
- To calculate change due for cash payments and log digital payment methods (UPI, Card).

- To record transaction history persistently and generate daily sales summaries and business analytics.

4. Project Description

SmartBill is structured as a modular Python billing application. When the program initiates, it verifies and loads the persistent product catalogue and invoice records. If launched for the first time, it automatically bootstraps the database with realistic sample items across multiple retail categories including Electronics, Groceries, Stationery, Beverages, and Personal Care.

The user is presented with an organized main menu comprising options: Create New Bill, Product Catalogue, Manage Inventory, Bill History, Sales Summary, Search Product, and Exit. During bill creation, the application captures customer details (Name and 10-digit Phone Number) and auto-generates a unique invoice number along with system date and time timestamps.

Cashiers add products to an active cart by specifying Product IDs. The system checks available shelf stock, enforces maximum quantity limits, and computes line subtotals. When checking out, the calculation engine evaluates item bulk discounts and store-wide bill offers, computes itemized GST on net taxable values, processes payments with exact change calculation, atomically updates inventory stock, and renders a professional receipt.

5. Concepts Used in the Program

Functions: The program is structured into 24 single-responsibility modular functions such as `find_product_by_id()`, `compute_bill_financials()`, `calculate_item_quantity_discount()`, `calculate_bill_tier_discount()`, `print_receipt()`, and `initialize_data_files()`.

Conditional Statements: Extensive `if`, `elif`, and `else` blocks are utilized to evaluate menu choices, discount thresholds, GST tax slabs, payment modes, and inventory boundary limits.

Loops: `while` loops sustain continuous menu interactions and input-validation prompts until valid entries are provided. `for` loops iterate over catalogue lists, cart items, and completed bill records for financial aggregations.

Exception Handling: Defensive `try-except` constructs intercept `ValueError` during numeric casting and handle `IOError / json.JSONDecodeError` during file operations, preventing runtime program termination.

Data Structures (Lists and Dictionaries): Python dictionaries model structured product entities and invoice records. Lists maintain the cart session, product collections, and historical ledgers.

Arithmetic & Financial Operators: Floating-point expressions compute line gross totals, tiered percentage discounts, taxable balances, GST collections, and cashier change returns.

String Formatting: Python f-strings with alignment and precision specifiers (e.g., `{price:>10.2f}`, `{name:<26}`) ensure professional receipt alignment and clean tabular console presentations.

5. Concepts Used in the Program (Continued)

File Handling & Data Persistence: Built-in `json` and `sqlite3` modules serialize and deserialize catalogue states and transaction history, ensuring that product modifications and completed sales persist across system restarts.

Date & Time Module: The standard `datetime` module generates live date stamps, time stamps, and formatted sequential invoice identifiers (`BILL-YYYYMMDD-XXXX`).

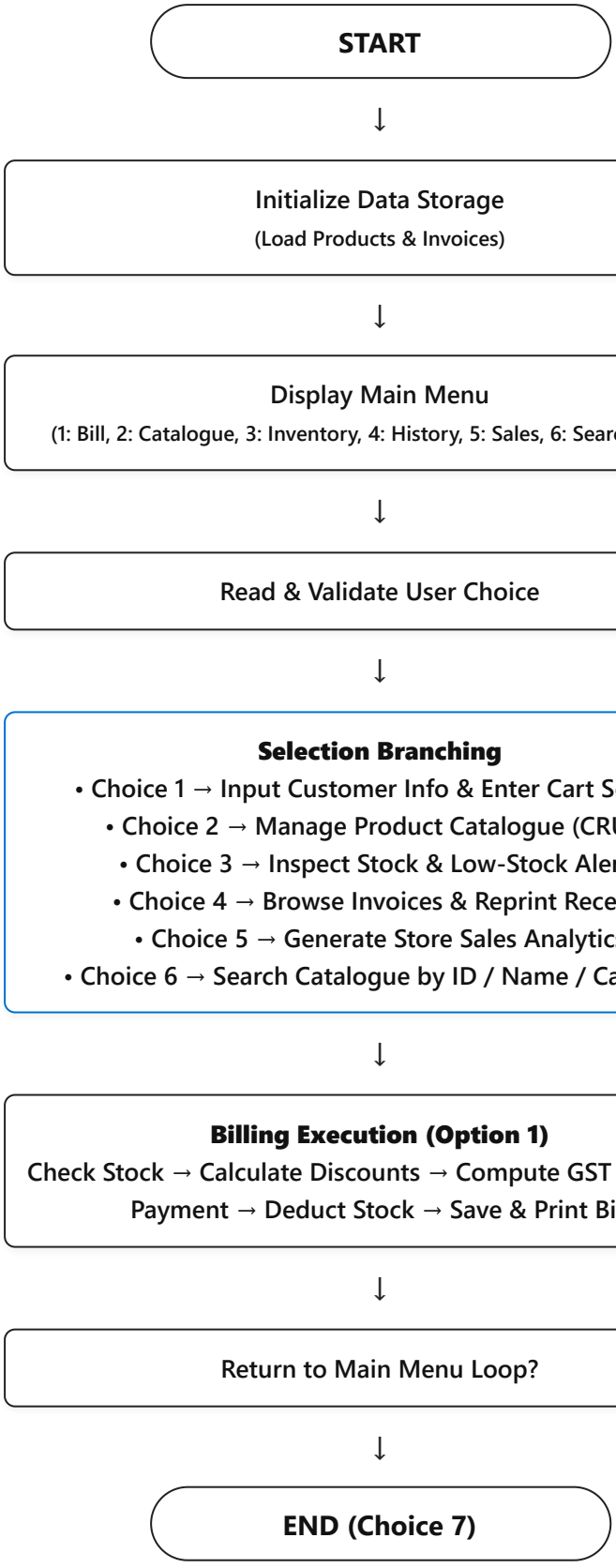
Validation & Defensive Programming: The system validates customer mobile numbers (must be exactly 10 digits), ensures non-negative prices, verifies positive non-zero quantities, and ensures tendered cash meets or exceeds the grand total.

6. Algorithm

1. Start the program and initialize data persistence files (`products.json` and `bills.json` / `database.db`).
2. Display the SmartBill Main Menu with options: Create Bill, Product Catalogue, Inventory, Bill History, Sales Summary, Search, and Exit.
3. Prompt the user to enter a menu choice (1–7).
4. **If Create New Bill is selected:**
   a. Prompt for customer name and 10-digit mobile number.
   b. Generate a unique invoice number, system date, and system time.
   c. Prompt for Product ID and search the catalogue. If not found or out of stock, notify the user.
   d. Prompt for quantity; verify that `requested_quantity ≤ available_stock`.
   e. Add item to cart and calculate item subtotal. Allow cashier to add more items, modify quantities, or remove items.
   f. When proceeding to checkout, compute item quantity discounts and overall bill-tier discounts.
   g. Distribute bill discounts proportionally across line items to calculate line-level taxable values.
   h. Compute GST per item based on its category GST rate (5%, 12%, 18%) and sum to find total GST.
   i. Calculate Grand Total: `Grand Total = Net Taxable Value + Total GST`.
   j. Prompt for payment method (Cash, UPI, Card). If Cash, input tender and compute `Change Due = Tender – Grand Total`.
   k. Atomically deduct purchased quantities from catalogue stock.
   l. Save the invoice record to persistent storage and print the formatted receipt.
5. **If Product Catalogue is selected:** Allow viewing all items, adding new products, modifying prices/stock, or deleting products.

6. **If Manage Inventory is selected:** Display stock levels, highlight low-stock items (≤ threshold), and allow quick restocking.
7. **If Bill History is selected:** Display historical transaction summaries and permit receipt reprinting.
8. **If Sales Summary is selected:** Aggregate total sales revenue, total GST collected, total discounts given, and top-selling products.
9. **If Search Product is selected:** Perform case-insensitive substring search by product name, category, or ID.
10. **If Exit is selected:** Display exit message and terminate program safely.

7. Flow Chart


START
Initialize Data Storage
(Load Products & Invoices)
Display Main Menu
(1: Bill, 2: Catalogue, 3: Inventory, 4: History, 5: Sales, 6: Search, 7: Exit)
Read & Validate User Choice
Selection Branching
• Choice 1 → Input Customer Info & Enter Cart Session
• Choice 2 → Manage Product Catalogue (CRUD)
• Choice 3 → Inspect Stock & Low-Stock Alerts
• Choice 4 → Browse Invoices & Reprint Receipt
• Choice 5 → Generate Store Sales Analytics
• Choice 6 → Search Catalogue by ID / Name / Category
Billing Execution (Option 1)
Check Stock → Calculate Discounts → Compute GST → Process Payment → Deduct Stock → Save & Print Bill
Return to Main Menu Loop?
END (Choice 7)

8. Python Program

Program Listing — Core Data Persistence & Discount Logic

```
001  import os, json, datetime
002
003  # Global File Paths and Currency Configuration
004  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
005  PRODUCTS_FILE = os.path.join(BASE_DIR, "products.json")
006  BILLS_FILE = os.path.join(BASE_DIR, "bills.json")
007  CURRENCY = "Rs."
008
009  def load_products():
010      """Load product catalogue from persistent JSON storage."""
011      try:
012          with open(PRODUCTS_FILE, "r", encoding="utf-8") as f:
013              return json.load(f)
014      except Exception:
015          return []
016
017  def save_products(products):
018      """Save product catalogue back to JSON file."""
019      with open(PRODUCTS_FILE, "w", encoding="utf-8") as f:
020          json.dump(products, f, indent=4)
021
022  def calculate_item_quantity_discount(price, qty):
023      """Apply item-level bulk purchase discount."""
024      subtotal = price * qty
025      if qty >= 10:
026          rate = 10.0
027      elif qty >= 5:
028          rate = 5.0
029      else:
030          rate = 0.0
031      discount_amt = round(subtotal * (rate / 100.0), 2)
032      return rate, discount_amt
033
034  def calculate_bill_tier_discount(net_subtotal):
035      """Apply tiered store-wide basket discount."""
036      if net_subtotal >= 5000:
037          rate = 15.0
038      elif net_subtotal >= 3000:
039          rate = 10.0
040      elif net_subtotal >= 1000:
041          rate = 5.0
042      else:
043          rate = 0.0
044      discount_amt = round(net_subtotal * (rate / 100.0), 2)
045      return rate, discount_amt
```

8. Python Program (Continued)

Program Listing — Financial Computation & Multi-Slab GST Calculation

```
046  def compute_bill_financials(cart):
047      """Calculates gross subtotal, discounts, taxable value, GST, and grand total."""
048      gross_subtotal = 0.0
049      total_qty_discount = 0.0
050      line_details = []
051
052      # Step 1: Compute item-level gross and quantity discounts
053      for item in cart:
054          line_gross = round(item["price"] * item["qty"], 2)
055          q_rate, q_disc = calculate_item_quantity_discount(item["price"], item["qty"])
056          line_after = line_gross - q_disc
057          gross_subtotal += line_gross
058          total_qty_discount += q_disc
059          line_details.append({"item": item, "gross": line_gross, "after": line_after, "disc": q_disc})
060
061      net_after_qty = gross_subtotal - total_qty_discount
062      b_rate, b_disc = calculate_bill_tier_discount(net_after_qty)
063      total_discount = round(total_qty_discount + b_disc, 2)
064      overall_taxable = round(gross_subtotal - total_discount, 2)
065
066      # Step 2: Proportional allocation of bill discount to compute itemized GST
067      total_gst = 0.0
068      processed_items = []
069      for line in line_details:
070          itm = line["item"]
071          share = round(b_disc * (line["after"] / net_after_qty), 2) if net_after_qty > 0 else 0.0
072          taxable = max(0.0, round(line["after"] - share, 2))
073          gst_amt = round(taxable * (itm["gst_rate"] / 100.0), 2)
074          total_gst += gst_amt
075          processed_items.append({
076              "id": itm["id"], "name": itm["name"], "price": itm["price"],
077              "qty": itm["qty"], "taxable": taxable, "gst_rate": itm["gst_rate"],
078              "gst_amt": gst_amt, "item_total": round(taxable + gst_amt, 2)
079          })
080
081      grand_total = round(overall_taxable + total_gst, 2)
082      return {
083          "subtotal": gross_subtotal, "qty_discount": total_qty_discount,
084          "bill_discount": b_disc, "total_discount": total_discount,
085          "taxable": overall_taxable, "gst": round(total_gst, 2),
086          "grand_total": grand_total, "items": processed_items
087      }
```

8. Python Program (Continued)

Program Listing — Input Helpers & Main Controller Loop

```
088  def input_int(prompt, min_val=None, max_val=None):
089      """Interactively prompts for a valid integer with error handling."""
090      while True:
091          try:
092              val = int(input(prompt).strip())
093              if min_val is not None and val < min_val:
094                  print(f"  [!] Minimum value is {min_val}."); continue
095              if max_val is not None and val > max_val:
096                  print(f"  [!] Maximum value is {max_val}."); continue
097              return val
098          except ValueError:
099              print("  [!] Invalid input. Please enter a valid integer.")
100
101  def main():
102      """Main interactive loop of SmartBill."""
103      initialize_data_files()
104      while True:
105          print("\n" + "=" * 50)
106          print("           SMARTBILL SYSTEM - MAIN MENU")
107          print("=" * 50)
108          print("  [1] Create New Bill\n  [2] Product Catalogue\n  [3] Manage Inventory")
109          print("  [4] Bill History\n  [5] Sales Summary\n  [6] Search Product\n  [7] Exit")
110          print("=" * 50)
111          choice = input("  Enter choice (1-7): ").strip()
112          if choice == '1': create_new_bill()
113          elif choice == '2': menu_product_catalogue()
114          elif choice == '3': menu_inventory()
115          elif choice == '4': menu_bill_history()
116          elif choice == '5': menu_sales_summary()
117          elif choice == '6': menu_search_product()
118          elif choice == '7': print("\nThank you for using SmartBill!"); break
```

9. Output

Sample Output 1 — Main Menu & Catalogue

```
================================================
           SMARTBILL SYSTEM - MAIN MENU
================================================
  [1] Create New Bill
  [2] Product Catalogue (View / Add / Edit / Remove)
  [3] Manage Inventory (Stock levels & Restocking)
  [4] Bill History (View Past Invoices & Reprint)
  [5] Sales Summary & Business Analytics
  [6] Search Product
  [7] Exit
================================================
  Enter choice (1-7): 2
```

Sample Output 2 — Product Catalogue Table with Stock Status

```
=============================================================================================
                                    PRODUCT CATALOGUE
=============================================================================================
ID     Product Name                 Category       Price(Rs.)   Stock   GST% Status
---------------------------------------------------------------------------------------------
P101   Wireless Optical Mouse       Electronics        499.00      25  18.0% OK
P102   Mechanical Gaming Keyboard   Electronics       1899.00      15  18.0% OK
P104   A4 Spiral Notebook (200p)    Stationery          85.00      50  12.0% OK
P106   Royal Basmati Rice (5 kg)    Groceries          450.00      20   5.0% OK
P110   Herbal Hand Sanitizer (500m  Personal Care      110.00       4  18.0% LOW (4)
=============================================================================================
```

Sample Output 3 — Generated Professional Customer Receipt

```
============================================================================
                           SMARTBILL
             SMART STORE BILLING & INVENTORY SYSTEM
             123 College Road, Silicon Corridor | Phone: +91 98765 43210
============================================================================
 Bill No  : BILL-20260908-0001           Date : 10/09/2026
 Customer : Priya Sharma                 Time : 11:35:00
 Phone    : 9876543210
----------------------------------------------------------------------------
 ID     Item Description            Qty    Price   GST%    Amount(Rs.)
----------------------------------------------------------------------------
 P101   Wireless Optical Mouse        5   499.00  18.0%        2517.20
 P104   A4 Spiral Notebook (200p)    10    85.00  12.0%         771.12
----------------------------------------------------------------------------
 Gross Subtotal                                  : Rs.    3345.00
 Quantity Bulk Discount                          : -Rs.    209.75
 Store Offer (10% off on bill)                   : -Rs.    313.53
 Total Discount Given                            : -Rs.    523.28
 Net Taxable Value                               : Rs.    2821.72
 Total GST Amount                                : Rs.     466.60
============================================================================
 GRAND TOTAL                                     : Rs.    3288.32
============================================================================
```

```
 Payment Mode : Cash
 Amount Paid  : Rs.  3500.00
 Change Due   : Rs.   211.68
================================================================================
                  Thank You for Shopping with SmartBill!
================================================================================
```

10. Conclusion

The **SmartBill – Smart Store Billing and Inventory Management System** successfully fulfills the academic objective of designing and developing an intelligent, robust, and real-world billing application in Python. Moving far beyond a basic calculator, the system demonstrates how automated store operations can eliminate manual pricing mistakes, enforce inventory constraints, and ensure statutory tax compliance.

The project highlights key Python programming principles including user-defined modular functions, exception handling, nested decision structures, persistent data serialization using JSON and SQLite, defensive input validation, and professional string formatting. The implementation of item-level bulk discounts alongside basket-value store offers provides a realistic commercial experience.

Real-time stock synchronization guarantees that shelf inventory automatically decrements upon successful checkout, preventing negative stock and providing early low-stock warnings to store managers. The application operates across both a fast command-line terminal and a modern browser-based web dashboard.

Overall, the project demonstrates how foundational programming techniques taught in the computer science curriculum can be synthesized to create an industry-relevant software product. Future enhancements can expand the platform with hardware barcode scanners, cloud database synchronization, and digital receipt dispatch via SMS or email.

Project Topic: SmartBill – Smart Store Billing and Inventory Management System

Student: J S Rahul

USN: 25BTRPW048

Class/Section: CSE-PWC-A

Faculty Guide: Ms. Sujithra G